

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

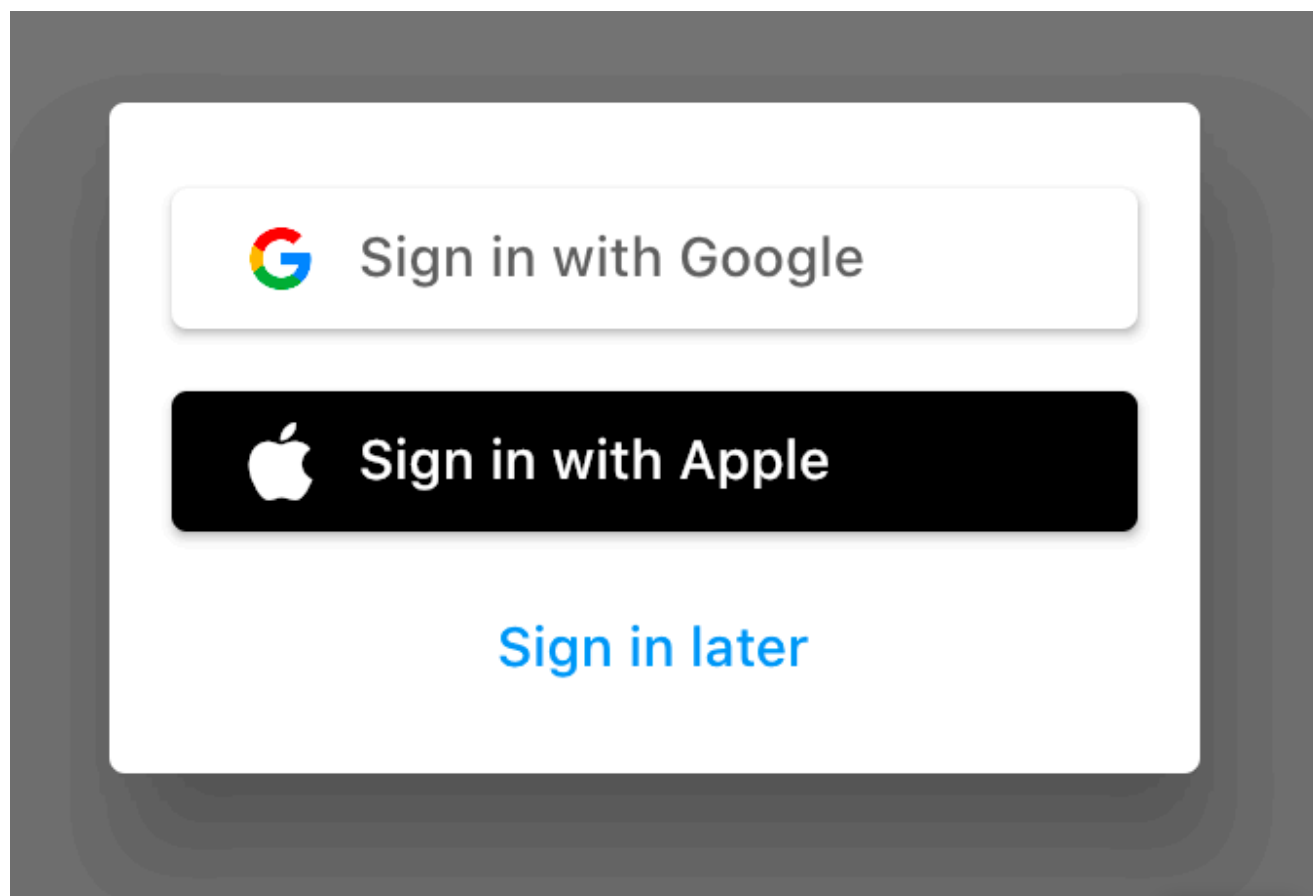
3..25.1

SETUP

 Copy as Markdown

Set up the legacy auth module

The legacy `serverpod_auth` module adds user management and sign-in with email, Google, Apple, and Firebase to existing Serverpod projects. New projects should use the [current authentication module](#) instead. To move an existing app off this module, see [Migrate from legacy auth](#). It is also possible to build a [custom authentication implementation](#).



Installing the auth module



Install the module on the server, client, and app as shown in the sections below. Beyond sign-in, the module handles basic user information, such as user names and profile pictures. Make sure to use the same version numbers as for Serverpod itself for all dependencies.

Server setup

Add the module as a dependency to the server project's `pubspec.yaml`.

```
$ dart pub add serverpod_auth_server
```

Add the authentication handler to the Serverpod instance.

```
import 'package:serverpod_auth_server/serverpod_auth_server.dart' as auth;

void run(List<String> args) async {
  var pod = Serverpod(
    args,
    authenticationHandler: auth.authenticationHandler, // Add this line
  );

  ...
}
```

Optionally, add a nickname for the module in the `config/generator.yaml` file. This nickname will be used as the name of the module in the code.

```
modules:
  serverpod_auth:
    nickname: auth
```

While still in the server project, generate the client code and endpoint methods for the auth module by running the `serverpod generate` command line tool.

```
$ serverpod generate
```

Initialize the auth database

After adding the module to the server project, you need to initialize the database. First you have to create a new migration that includes the auth module tables. This is done by running the `serverpod create-migration` command line tool in the server project.

```
$ serverpod create-migration
```

Start your database container from the server project.

```
$ docker compose up --build --detach
```

Then apply the migration by starting the server with the `apply-migrations` flag.

```
$ dart run bin/main.dart --role maintenance --apply-migrations
```

The full migration instructions can be found in the [migration guide](#).

Configure Authentication

The module comes with a default authentication configuration. To customize it, go to your main `server.dart` file, import the `serverpod_auth_server` module and set up the authentication configuration:

```
import 'package:serverpod_auth_server/serverpod_auth_server.dart' as auth;

void run(List<String> args) async {

  auth.AuthConfig.set(auth.AuthConfig(
    minPasswordLength: 12,
  ));

  // Start the Serverpod server.
  await pod.start();
}
```

Property	Description	Default
allowUnsecureRandom	True if unsecure random number generation is allowed. If set to false, an error will be thrown if the platform does not support secure random number generation.	false
appleClientIds	The client identifiers Sign in with Apple accepts identity tokens for: your app's bundle IDs and any services IDs. Sign in with Apple is disabled until this is set.	<code>{}</code>
appleIdentityTokenClockSkewTolerance	How much clock drift to tolerate when checking the expiry and issue time of a Sign in with Apple identity token.	1min

Property	Description	Default
emailSignInFailureResetTime	The reset period for email sign in attempts. Defaults to 5 minutes.	5min
enableUserImages	True if user images are enabled.	true
extraSaltyHash	True if the server should use the accounts email address as part of the salt when storing password hashes (strongly recommended).	true
firebaseServiceAccountKeyJson	Path to the Firebase service account key JSON file. Generate and download from the Firebase console.	<code>config/firebase_service_account_key.json</code>
importUserImagesFromGoogleSignIn	True if user images should be imported when signing in with Google.	true
maxAllowedEmailSignInAttempts	Max allowed failed email sign in attempts within the reset period.	5
maxPasswordLength	The maximum length of passwords when signing up with email.	128
minPasswordLength	The minimum length of passwords when signing up with email.	8
onUserCreated	Called after a user has been created. Listen to this callback if you need to do additional setup.	-
onUserUpdated	Called whenever a user has been updated. This can be when the user name is changed or if the user uploads a new profile picture.	-
onUserWillBeCreated	Called when a user is about to be created, gives a chance to abort the creation by returning false.	-

Property	Description	Default
passwordResetExpirationTime	The time for password resets to be valid. Defaults to 15 minutes.	15min
sendPasswordResetEmail	Called when a user should be sent a reset code by email.	-
sendValidationEmail	Called when a user should be sent a validation code on account setup.	-
userCanEditFullName	True if users can edit their full name.	false
userCanEditUserImage	True if users can update their profile images.	true
userCanEditUserName	True if users can edit their user names.	true
userCanSeeFullName	True if users can view their full name.	true
userCanSeeUserName	True if users can view their user name.	true
userImageFormat	The format used to store user images.	jpg
userImageGenerator	Generator used to produce default user images.	<code>defaultUserImageGenerator</code>
userImageQuality	The quality setting for images if JPG format is used.	70
userImageSize	The size of user images.	256

Property	Description	Default
userInfoCacheLifetime	The duration which user infos are cached locally in the server.	1min
validationCodeLength	The length of the validation code used in the authentication process. This value determines the number of digits in the validation code. Values below 8 log a security warning, and values below 4 throw an <code>ArgumentError</code> .	8

Client setup

Add the auth client in your client project's `pubspec.yaml`.

```
dependencies:
  ...
  serverpod_auth_client: 4.0.0
```

App setup

First, add dependencies to your app's `pubspec.yaml` file for the methods of signing in that you want to support.

```
dependencies:
  flutter:
    sdk: flutter
  serverpod_flutter: 4.0.0
  auth_example_client:
    path: ../auth_example_client

  serverpod_auth_shared_flutter: 4.0.0
```

Next, you need to set up a `SessionManager`, which keeps track of the user's state. It will also handle the authentication keys passed to the client from the server, upload user profile images, etc.

```
late SessionManager sessionManager;
late Client client;

void main() async {
  // Need to call this as we are using Flutter bindings before runApp is called.
  WidgetsFlutterBinding.ensureInitialized();

  // The android emulator does not have access to the localhost of the machine.
  // const ipAddress = '10.0.2.2'; // Android emulator ip for the host
```

```
// On a real device replace the ipAddress with the IP address of your computer.
const ipAddress = 'localhost';

// Sets up a singleton client object that can be used to talk to the server from
// anywhere in our app. The client is generated from your server code.
// The client is set up to connect to a Serverpod running on a local server on
// the default port. You will need to modify this to connect to staging or
// production servers.
client = Client('http://$ipAddress:8080/')
  ..authKeyProvider = FlutterAuthenticationKeyManager()
  ..connectivityMonitor = FlutterConnectivityMonitor();

// The session manager keeps track of the signed-in state of the user. You
// can query it to see if the user is currently signed in and get information
// about the user.
sessionManager = SessionManager(
  caller: client.modules.auth,
);
await sessionManager.initialize();

runApp(MyApp());
}
```

The `SessionManager` has useful methods for viewing and monitoring the user's current state.

Check authentication state

To check if the user is signed in:

```
sessionManager.isSignedIn;
```

Returns `true` if the user is signed in, or `false` otherwise.

Access current user

To retrieve information about the current user:

```
sessionManager.signedInUser;
```

Returns a `UserInfo` object if the user is currently signed in, or `null` if the user is not.

Register authentication

To register a signed in user in the session manager:

```
await sessionManager.registerSignedInUser(
```

```
userInfo,  
keyId,  
authKey,  
);
```

This will persist the user information and store the auth key in the client's key manager. For more details, see [Custom Providers - Client Setup](#).

Monitor authentication changes

To add a listener that tracks changes in the user's authentication state, useful for updating the UI:

```
@override  
void initState() {  
  super.initState();  
  
  // Rebuild the page if authentication state changes.  
  sessionManager.addListener(() {  
    setState(() {});  
  });  
}
```

The listener is triggered whenever the user's sign-in state changes.

Sign out current device

To sign the user out on from the current device:

```
await sessionManager.signOutDevice();
```

Returns `true` if the sign-out is successful, or `false` if it fails.

Sign out all devices

To sign the user out across all devices:

```
await sessionManager.signOutAllDevices();
```

Returns `true` if the user is successfully signed out from all devices, or `false` if it fails.



The `signOut` method was removed in the 3.0 release of the module. Use `signOutDevice` or `signOutAllDevices` instead.